

Base de données NoSQL

Bases

Louis Roussel

louis.rousseau@univ-lille.fr

Support de cours par **Bastien Cazaux** qui s'appuie sur les cours d'**Anne Etien** et de **Anne Cécile Caron** de l'université de Lille et celui de **Régis Behmo** et **Nicolas Travers** de CentralSupélec.

Plan du cours

Plan du cours		
Un peu d'histoire ...	3	
Un peu d'histoire : les années 80		
Un peu d'histoire : les années 90		
Un peu d'histoire : milieu des années 2000		
NoSQL Définition	6	
NoSQL		
Différent modèles de données		
Avant de commencer : à vous de travailler		
Bases de données agrégat	9	
Clé-Valeur		
Exercice : Application à la famille		
Clé-Valeur		
Clé-Valeur : Forces, Faiblesses et Cas d'usage		
Famille de colonnes		
Exercice : Application à la famille de colonnes		
Famille de colonnes : Forces, Faiblesses et Cas d'usage		
Document		
Exercice : Application à la famille de documents		
Document : Forces, Faiblesses et Cas d'usage		
BD orienté agrégat		
Résumé des bases de données orientées agrégat		
Bases de données graphe	20	
Relations		
Base de donnée graphe		
Exercice : Application à la famille de graphes		
BD graphe : Forces, Faiblesses et Cas d'usage		
Quelques BD en pratique et utilisations		
Base de données sans schéma	26	
Différence BDR - NoSQL		
Différence BDR - NoSQL		
Sans schéma		
Schéma implicite		
Schéma implicite : problèmes		
Schéma implicite : réduire les problèmes		

- Milieu 1980s : Effervescence des bases de données relationnelles :
 - Avantages :
 - Persistance : mécanisme de sauvegarde et restauration des données
 - Concurrence : plusieurs applications accèdent à des données partagées. Grâce aux transactions, possibilité de coordonner des interactions concurrentielles
 - Intégration : plusieurs applications stockent leurs données dans la même base
 - Un modèle (presque) standard : les dialectes SQL sont similaires
 - Inconvénients :
 - « Impedance Mismatch » : les données sont sauvegardées sous forme de tuples alors que la structure des données en mémoire est plus complexe (par exemple des objets)
 - Besoin de faire une traduction entre les deux représentations
 - Les ORM (mapping objet-relationnel) comme JPA font cette traduction MAIS besoin de décomposer un concept complexe en plusieurs petits et à l'inverse de le reconstruire lors de l'interrogation.

- Milieu 1990s : Naissance des bases de données relationnelles objet (ORDB)
 - Principe : Réduire l'écart entre la structure des données dans l'application (objet) et dans la base de données.
 - Mais +/- échec : les bases de données relationnelles ont accentué leur rôle de mécanisme d'intégration (plusieurs applications autour de la même base de données).
- **Domination du relationnel pendant 20 ans**
- **Mais** une structure définie pour intégrer plusieurs applications finit par devenir complexe
 - les changements de structure requis par une application impactent les autres
 - les applications ont des besoins différents en terme de performance
 - une base gérée par plusieurs équipes (une par application) mais il faut avoir confiance dans la façon dont les données sont sauvegardées VS une base par application gérée par une équipe.

- Milieu des années 2000 : Avènement d'internet
 - Beaucoup de trafic (Google, Amazon, ...)
 - Augmentation de la taille du serveur
 - Augmentation du nombre de serveurs
 - Or, le relationnel a été défini pour travailler sur un (gros) serveur et ne marche pas sur les clusters.
 - Création par Google (BigTable) et Amazon (Dynamo) d'alternatives au relationnel pour fonctionner sur des clusters (rapidement suivi par d'autres entreprises).
 - #NoSQL est d'abord un hashtag twitter (court et clair)
 - pour parler des nouveaux types de BD lors d'un meetup en 2009 à San Francisco
 - devenu ensuite le nom d'un mouvement

NoSQL

- Pas de définition précise
 - Et certainement pas *Not Only SQL* (sinon Postgres et Oracle sont inclus)
- Mais des caractéristiques :
 - N'utilise pas SQL mais d'autres langage de requêtes
 - CQL (Cassandra Query Language) "exactly like SQL (except where it's not)"
 - En fait **non relationnel**
 - Souvent **open source**
 - Adapté à la culture web du 21^{ème} siècle
 - Sans schéma
 - donc plus de flexibilité
- Persistance polyglotte (différentes technologies de stockage de données)
- Deux raisons principales d'utiliser NoSQL :
 - Besoin de gérer un gros volume de données de façon efficace
 - Améliorer la productivité de développement d'une application en utilisant un style d'interaction de données plus adapté

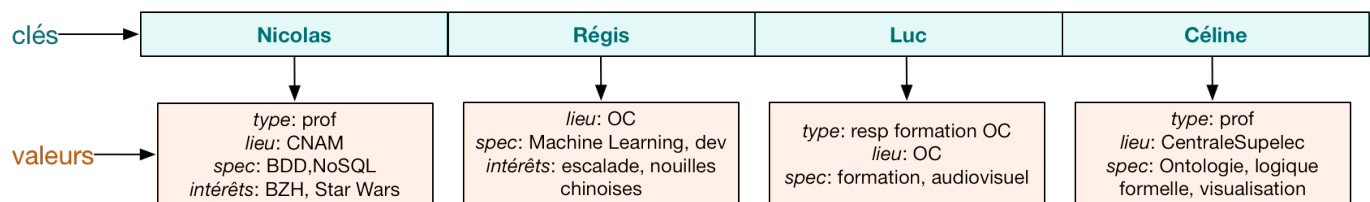
- On va parler dans la suite de 4 grandes familles de base de données NoSQL :
 - Clé-Valeur
 - Redis, Memcached, Azure Cosmos DB, SimpleDB
 - Document
 - MongoDB, CouchBase, DynamoDB, Cassandra
 - Famille de colonnes
 - BigTable, HBase, Spark SQL, Elasticsearch
 - Graphe
 - Neo4j, OrientDB, FlockDB
- Chacune de ces familles répond à des besoins très spécifiques
- Choix de la base de données NoSQL en fonction de votre application

Avant de commencer : à vous de travailler

- Mettez vous par groupe pour former 5 ou 6 groupes
- Chaque groupe choisit un film et fait un petit récapitulatif du film :
 - titre, genre, acteurs, pays d'origine, ...
- Un étudiant de chaque groupe vient au tableau pour écrire le récap. du film

Vous avez 5 minutes !

- Système le plus simple
- A partir d'une clé on récupère une valeur
 - hash map
- La valeur peut être
 - Un simple nom,
 - Une image,
 - Un document, ...
- La BD ne sait pas ce qu'est la valeur et s'en moque
 - Pas de possibilité de faire des requêtes dessus mais juste y accéder
 - CRUD opérations (Create, Read, Update, Delete)
- Système assez opaque



Exercice : Application à la famille Clé-Valeur

- Forces :
 - Très rapide
 - Passage à l'échelle
 - Modèle simple (Distribution horizontale très facile)
- Faiblesses
 - Interrogation seulement sur clé
 - Modèle de données trop pauvre pour les données complexes
 - Déporte une partie de la complexité au niveau de l'application
- Adapté pour les applications "data intensive" :
 - Le stockage des données avec identifiant unique :
 - sessions, profil, préférences utilisateur – panier d'achat
 - données avec des besoins de requêtage très simples avec des temps de réponse très rapides
- Non adapté pour :
 - relations entre données, corrélations entre diff. ens. de clés
 - opérations sur des ensembles (opérations limitées sur une clé à la fois – déporté au niveau du client)
 - requêtes sur les données autre que la clé
 - transactions sur des opérations multiples

Famille de colonnes

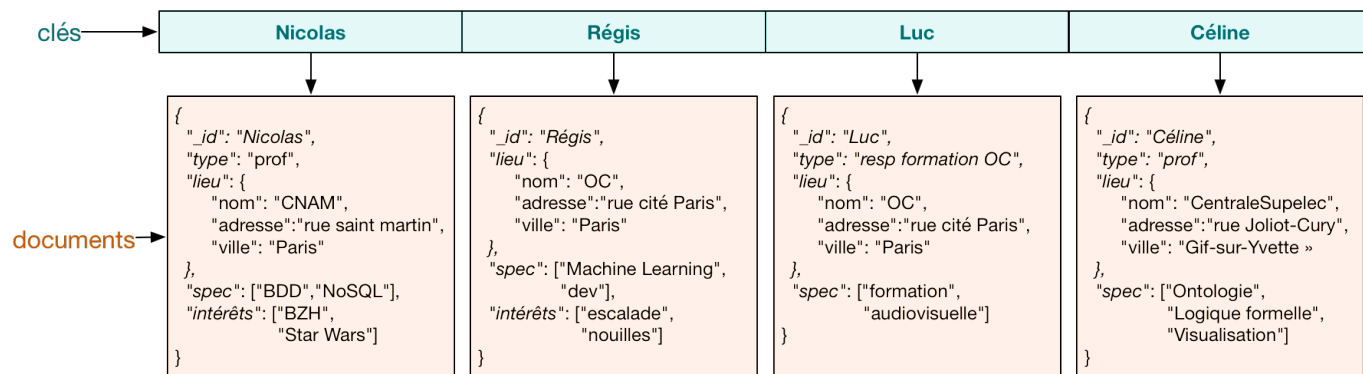
- Famille de colonnes = structure de clés à deux niveaux
- Première clé
 - souvent = identifier de ligne
 - est associé à une *map de valeurs plus détaillées appelées colonnes*
- Colonnes organisées en famille de colonnes
 - Chaque colonne doit être dans une seule famille
 - Chaque colonne agit comme unité d'accès, mais avec la supposition que les données d'une même famille vont être habituellement accédées ensemble

Stockage orienté lignes					Stockage orienté colonnes							
id	type	lieu	spec	intérêts	id	type	id	lieu	id	spec	id	intérêts
Nicolas	prof	CNAM	BDD, NoSQL	BZH, Star Wars	Nicolas	prof	Céline	Centrale Supelec	Nicolas	BDD	Nicolas	BZH
Régis		OC	Machine Learning, Dev	escalade, nouilles chinoises	Céline	prof	Nicolas	CNAM	Nicolas	NoSQL	Nicolas	Star Wars
Luc	resp formation OC	OC	formation, audiovisuel		Luc	resp formation OC	Régis	OC	Régis	Machine Learning	Régis	escalade
Céline	prof	CentraleSupelec	Ontologie, logique formelle, visualisation				Luc	OC	Régis	Dev	Régis	nouilles chinoises
									Luc	formation		
									Luc	audiovisuel		
									Céline	Ontologie		
									Céline	logique formelle		
									Céline	visualisation		

Famille de colonnes : Forces, Faiblesses et Cas d'usage

- Forces
 - modèle de données plus riche
 - scalabilité horizontale et utilisation de *MapReduce*
 - résultat des requêtes en temps réel
- Faiblesses
 - difficile à utiliser pour les données interconnectés
 - maintenance lors de l'ajout/suppression de colonnes
 - ne pas utiliser pour les requêtes non temps réel et inconnues
- A utiliser pour
 - la journalisation d'évènements
 - les compteurs (analytique sur le web)
 - applications avec beaucoup d'écriture et peu de lectures, priorité à la disponibilité
 - analyse en temps réel (cerner l'audience)
- A ne pas utiliser pour
 - applications avec des besoins de transactions ACID
 - agrégation par requêtes (sera effectuée par l'application client)

- Document = structure arborescente de clés
- A partir d'une clé on récupère un nouveaux document
- La structure du document est connue (JSON, XML, ...)



- Possibilité de faire des requêtes pour récupérer tout ou partie du document
- **Schéma implicite**

Exercice : Application à la famille de documents

- Forces
 - modèle de données simple mais puissant
 - bon passage à l'échelle
 - forte expressivité pour les requêtes (sur des structures imbriquées)
- Faiblesses
 - inadapté pour les données interconnectées
 - peut être lent pour les requêtes avec *MapReduce*
- A utiliser pour
 - Systèmes de gestion de contenu (profils utilisateurs, avis et commentaires, ...)
 - gestion d'évènements
 - applications de e-commerce (stockage du catalogue, schéma flexible pour les produits et commandes)

BD orienté agrégat

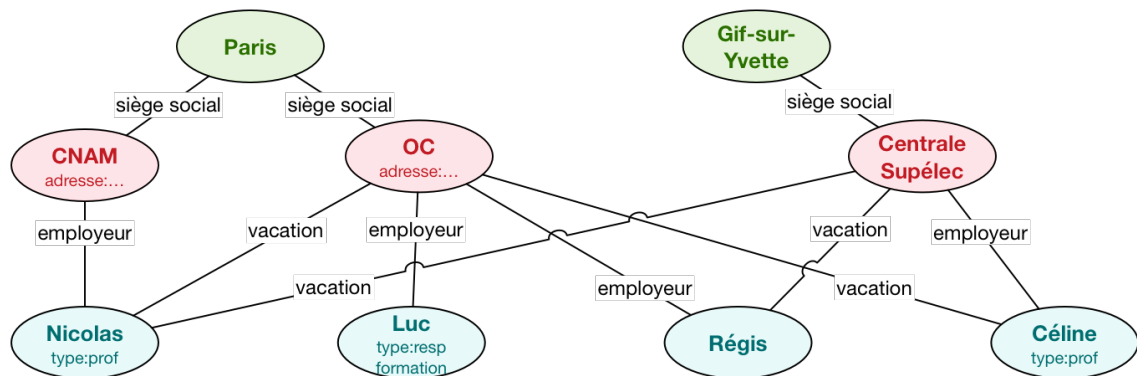
- Clé-Valeur, Famille de colonnes et document ne sont pas si différents
 - Utilisation de metadata (valeurs vers des références à d'autres objets) dans les clé-valeurs
 - Utilisation d'un *id* dans les familles de colonnes pour accéder aux autres colonnes
 - Utilisation d'un *id* dans les documents pour récupérer les données pour cet id
- Agrégat = concept éventuellement complexe
 - vient du domain driven design
 - = collection d'objets reliés que l'on souhaite considérer comme une unité
 - Unité pour la manipulation de données et la consistance

- Points communs
 - Un agrégat est indexé par une clé
 - Toutes les données d'un agrégat sont sur le même noeud du cluster
 - L'agrégat est l'unité atomique pour les mises à jours
- Différences
 - Clé-Valeur : l'agrégat est opaque, pas de possibilité de retrouver une partie
 - Document : l'agrégat est transparent mais sans schéma pas de possibilité d'optimiser le stockage ou la recherche de parties
 - Famille de colonnes : l'agrégat est divisé en famille de colonnes pouvant être traitées comme unité de données dans les lignes agrégats. Il y a donc une structure dans les données, mais la BD peut en tirer partie.

Relations

- Différents type de relations existent
 - L'agrégat :
Correspond à une inclusion d'un objet dans un autre
Limite
 - Atomicité supporté uniquement au sein d'un agrégat
Si MAJ de plusieurs agrégats en même temps : le développeur doit faire les vérifications (par opposition au BDR où ACID assure la modification de plusieurs lignes en une fois).
 - La relation :
Correspond à une référence d'un objet sur un autre
Utilisation
 - Base de données relationnelles
Mais pas idéal non plus à cause des jointures
 - Base de données orientée graphe

- BD agrégat répondent au besoin de fonctionner sur un cluster
- BD graphe répondent à une autre frustration :
 - Petits enregistrements avec beaucoup d'interconnexions
 - Propose un nouveau modèle
- Graphe = noeuds + arcs
 - Avec variations suivant ce qui est stocké dans les noeuds et dans les arcs
 - Dans FlockDB : noeuds et arcs simple sans attributs
 - Dans Neo4j : possibilité d'attacher des objets Java aux noeuds ou aux arcs



Exercice : Application à la famille de graphes

- Forces
 - modèle de données puissant
 - rapide pour les données liées
 - modèles d'interrogation performants : *Cypher*, *SPARQL*
 - Algorithmes de la théorie des graphes (le plus court chemin, ...)
- Faiblesses
 - Fragmentation, possible seulement pour certains domaines
 - Doit traverser le graphe entier avant d'obtenir une réponse définitive
 - Les transactions ACID doivent couvrir plusieurs noeuds et arcs pour maintenir la consistance
- A utiliser pour
 - Applications avec des données connectées
 - Applications de routage ou basées sur la localisation
 - Moteurs de recommandation
 - Business Intelligence
- Plus adapté pour fonctionner sur un seul serveur que sur un cluster

1. transactions rapides
2. gestion de contenu (bibliothèques numériques, collections de produits, dépôts de logiciels, collections multimédia, etc.)
3. collection d'événements complexes
4. e-commerce
5. web social (Linked Data)
6. chat
7. Comptage (vote en ligne, compteur, etc)
8. réseaux sociaux (recommandation, plus court chemin, cluster...)
9. recherche de produits dans une catégorie
10. gestion de cache
11. reporting à large échelle
12. détection de fraude en temps réel
13. framework stockant des objets
14. gestion des historiques d'utilisateurs sur réseaux sociaux
15. journalisation
16. réseaux SIG (routes, réseau électrique, fret...)
17. fichiers de logs

BD Clé-Valeur

- **Redis** : Vodafone, Trip Advisor, Nokia, Samsung, Docker
- **Memcached** : LiveJournal, Wikipédia, Flickr, Wordpress
- **Azure Cosmos DB** : Real Madrid, Orange tribes, MSN, LG, Schneider Electric
- **SimpleDB** : Amazon

BD Famille de colonnes

- **BigTable, HBase, Spark SQL, Elasticsearch** : moteur de recherche

BD Document

- **MongoDB** : ADP, Adobe, Bosch, Cisco, eBay, Electronic Arts, Expedia, Foursquare
- **CouchBase** : AOL, AT&T, Comcast, Disney, PayPal, Ryanair
- **DynamoDB SQL** : BMW, Dropcam, Duolingo, Supercell, Zynga
- **Cassandra** : NY Times, eBay, Sky, Pearson Education

BD Graphe

- **Neo4j** : eBay, Cisco, UBS, HP, TomTom, The National Geographic Society
- **OrientDB** : Comcast, Warner Music Group, Cisco, Sky, United Nations, VErisign
- **FlockDB SQL** : Twitter

Différence BDR - NoSQL

- En BD relationnelles :
 1. Définir un schéma
Quelles tables existent avec quelles colonnes, de quel type, ...
 2. Stocker les données
- En NoSQL :
 - Avec BD Clé-Valeur : stockage de n'importe quelle valeur par clé
 - Avec BD Document : idem
 - Avec BD Famille de colonnes : stockage de n'importe quelle donnée dans n'importe quelle colonne
 - Avec BD Graphe : possibilité d'ajouter des noeuds et des arcs comme on veut

- Avec un schéma :
 - Besoin d'imaginer à l'avance ce dont on a besoin de stocker
 - Evolution du schéma rare et souvent complexe
 - Données homogènes
- Sans schéma :
 - On peut facilement stocker tout ce qu'on veut
 - Possibilité de changer le stockage des données facilement
 - Ajouter ou supprimer des choses est simple
 - Gestion plus simple des données non uniformes
 - Chaque enregistrement contient uniquement le nécessaire, ni plus, ni moins

Sans schéma

- Très pratique si :
 - On stocke les données et
 - On les affiche dans un rapport comme une simple liste
- MAIS souvent on veut faire plus via des programmes
 - Qui ont besoin de savoir par exemple que :
 - l'adresse de facturation s'appelle `adresseFacturation` et non `facturationAdresse`
 - ou que le champ `quantite` va être un entier 5 et non la chaîne `five`
- Quand on écrit un programme qui accède aux données, il repose sur un **schéma implicite**

- Les programmes utilisant la base de données ont besoin de savoir :
 - Que certains champs existent
 - Quelle est la sémantique du champ
 - Quel est le type de la donnée stockée
- Le programme ne peut pas inférer que `qte` et `quantite` peuvent faire référence à la même chose
- Schéma Implicite = ensemble de suppositions sur la structure des données qui se retrouvent dans le code manipulant ces données

Schéma implicite : problèmes

- Avoir le schéma de façon implicite dans le code de l'application implique :
 - Besoin de creuser dans le code de l'application pour comprendre ce que représente la donnée
 - Plus ou moins facile selon la structuration du code
 - La base de données reste ignorante du schéma
 - La BD ne peut pas s'appuyer sur le schéma pour décider comment stocker ou retrouver des données
 - Pas de validation par la BD elle-même
- **Une base de données sans schéma déplace le schéma dans le code de l'application qui l'accède**
 - Problématique si plusieurs applications développées par des équipes différentes accèdent à la même base
 - Difficulté pour l'évolution de la base afin de pouvoir accéder aux anciennes comme aux nouvelles données

- Plusieurs solutions existent :
 - Encapsuler toutes les interactions avec la BD dans une seule application
 - Et utiliser des web services pour l'intégrer aux autres applications
 - Clairement délimiter les différentes parties d'un agrégat en fonction des différentes applications qui l'accèdent
 - Différentes sections dans une BD document
 - Différentes familles de colonnes dans une BD famille de colonnes